S.T.T.A PHASE-3

TEAM NUMBER - 83

1) STEP-1 (Mapping ER to relational model)

For this stage, I performed a direct, one-to-one translation of the conceptual **Entity-Relationship (ER) model** into an initial **logical relational schema**. The goal was to create a foundational structure that mirrors the conceptual design *exactly*, before any normalization or relational rules are applied.

The process was as follows:

1. **Mapping Entities to Relations:** Each entity set from the ER diagram was directly converted into a **relation** (a table).
2. **Mapping Attributes to Columns:** All attributes of an entity, regardless of their type, were mapped to their own corresponding **columns** in that table.
    - **Simple** attributes became single columns.
    - **Composite** attributes were also mapped to a *single column,* preserving their composite structure at this stage.
3. **Mapping Relationships via Keys:** Relationships between entities were implemented using **foreign keys** to link the tables.
    - For **1:N** (one-to-many) relationships, the primary key of the "one" side was added as a foreign key to the "many" side's table.

- For **N:M** (many-to-many) relationships, a new **junction table** was created, containing the primary keys of both entities to link them.

2) STEP-2 (Converting Relational model to 1NF)

To normalize the database schema to 1NF, we focused on ensuring that all attributes are **atomic** and that there are no **repeating groups** or **multi-valued attributes**.

Here are the specific transformations that were made:

1. Resolving Multi-valued Attributes

- **Problem:** The `Known_Aliases` attribute was multi-valued. A single record could store a list of multiple aliases (e.g., "Alias A", "Alias B") within a single cell.
- **Solution:** This was resolved by creating a new, separate table " Individual_Aliases ". This new table holds each alias in its own row, along with a foreign key that links back to the primary key of the original entity. This eliminates the repeating group and ensures each cell contains only one value.

2. Decomposing Composite Attributes

- **Problem:** Several attributes were **composite**, meaning they stored multiple distinct pieces of information. This violates the principle of atomicity.
- **Solution:** These composite attributes were broken down into their smallest, indivisible (atomic) components:
  - The `Name` attribute was decomposed into `FirstName` and `LastName`.
  - The `Coordinates` attribute was decomposed into `Latitude` and `Longitude`.

By implementing these changes, every attribute in the tables now holds a single, atomic value, satisfying the requirements of First Normal Form.

3) STEP-3 (Converting Relational model to 2NF)

To advance from 1NF to 2NF, the model must be in 1NF and must have all **partial dependencies** removed. A partial dependency exists when a non-key attribute is dependent on only a portion of a composite primary key.

- **Analysis:** We analyzed all tables in the 1NF model that use a **composite primary key**.
- **Observation:** In every case, it was found that all non-key attributes are **fully functionally dependent** on the *entire* composite key, not just a part of it.
- **Conclusion:** Since no partial dependencies were identified, the relational model already satisfies the requirements of 2NF. No changes or further decomposition were necessary.

4) STEP-4(Converting Relational model to 3NF)

To satisfy 3NF, we must ensure that all non-key attributes are directly dependent on the primary key, and not on any other non-key attribute. This process involves identifying and removing any **transitive dependencies**.

1. Identifying the Transitive Dependency

• In the `Individual` table, the primary key is `Individual_ID`.

• We observed the following functional dependencies:

  ◦ `Individual_ID` → `Date_of_Birth` (A person's ID determines their date of birth)

  ◦ `Date_of_Birth` → `Age` (A person's age is determined by their date of birth)

• This creates a **transitive dependency**: `Individual_ID` → `Date_of_Birth` → `Age`. The non-key attribute `Age` is dependent on another non-key attribute, `Date_of_Birth`, rather than directly and only on the primary key.

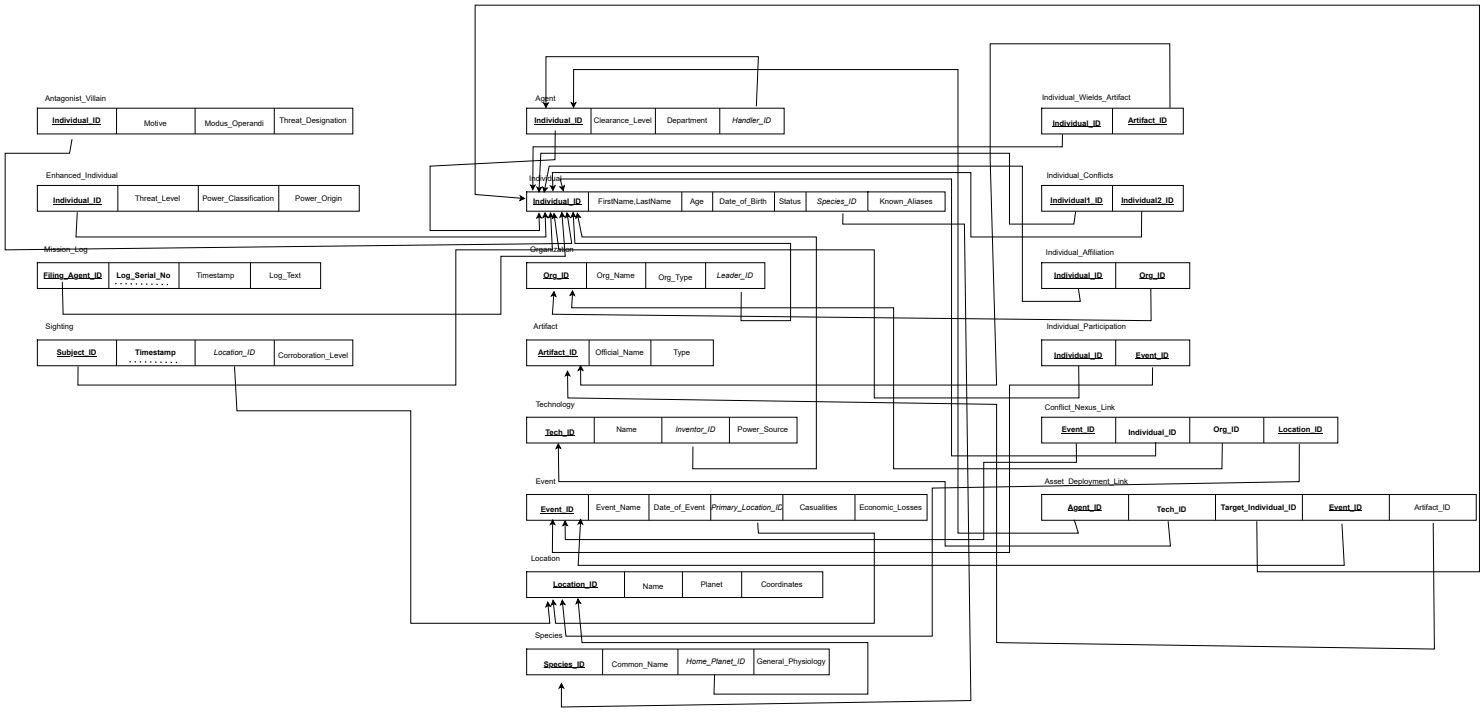
2. Resolving the Dependency

• **Problem:** The `Age` attribute violates 3NF. It also represents a **calculated field**, as its value is not static and can always be derived from the `Date_of_Birth` and the current date. Storing calculated data leads to redundancy and potential update anomalies (e.g., the age would need to be updated every day for every record).

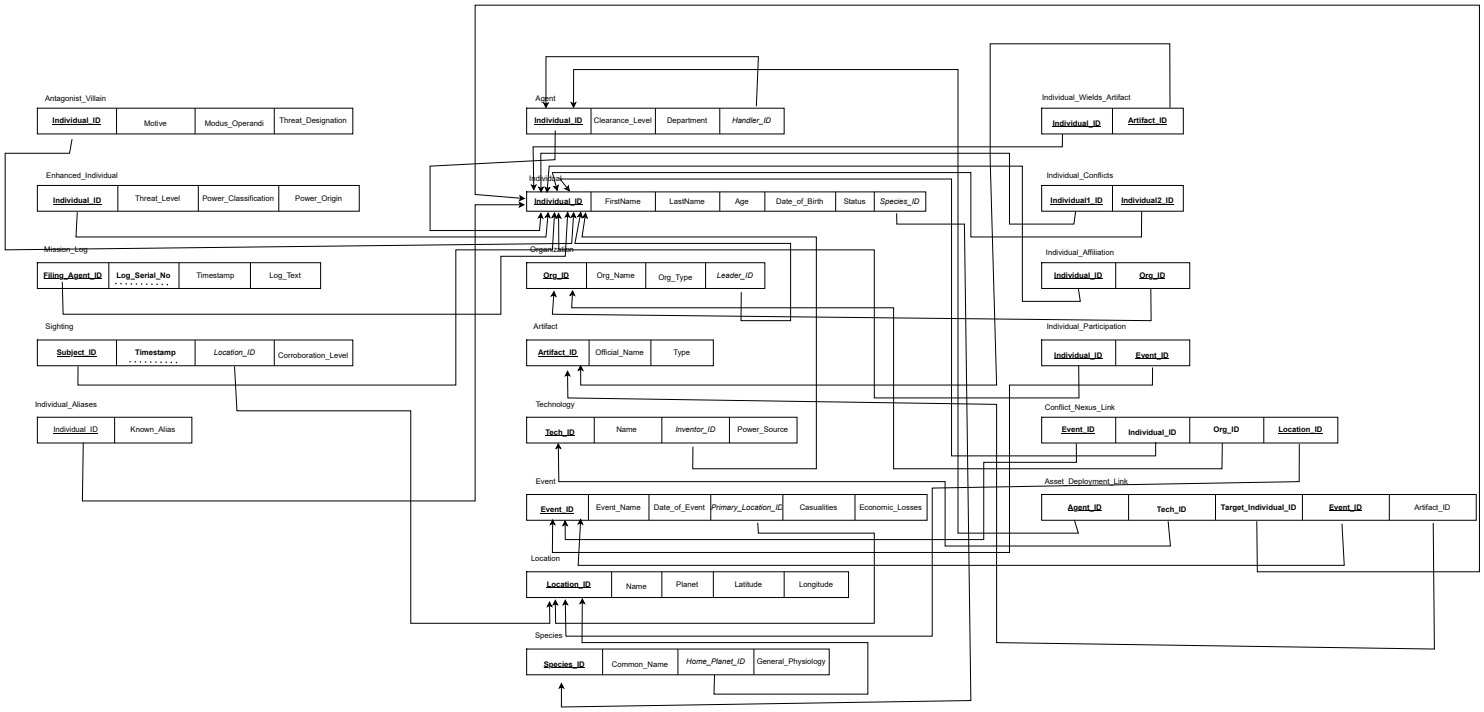
• **Solution:** The `Age` attribute was simply **removed** from the `Individual` table.

• **Outcome:** By removing this transitively dependent attribute, the `Individual` table now satisfies 3NF. All remaining non-key attributes (like `Date_of_Birth`) are fully and directly dependent on the primary key (`Individual_ID`). The `Age` can be easily calculated in any application or query when needed, ensuring the data is always accurate.

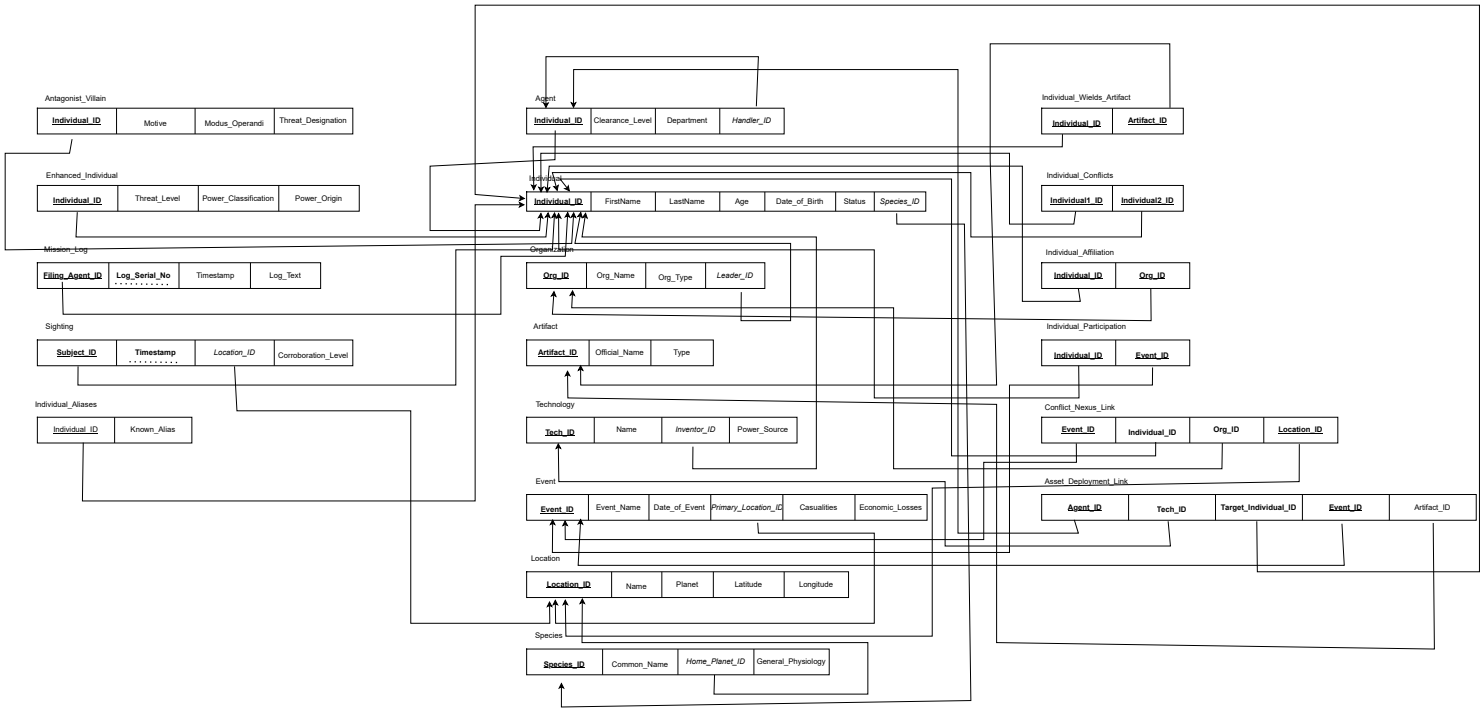
1.After mapping ER to relational model



2.Relational model after conversion to 1NF



3.Relational model after conversion to 2NF



4.Relational model after conversion to 3NF